Javascript

Page No.
Date

★ Data types and Truthy false values.

Primitive Datatypes:-

① Number :- it can be (int or float) — let age = 21; let price = 99.9;

② String = Represent text enclosed " " — let name = "Atharva";

③ Boolean = Represent True / false — let isLogin = true;

④ Null = Represent empty — let x = null;

⑤ BigInt = Represent big numbers — let bigNum = 1234567890123456789012345678904n;

Non primitive datatype:-

① Object :- key-value pair :- let person = {name: "Atharva", age: 24};

② Array :- ordered list of value :- let nums = [1,2,3,4];

③ function :- Reusable code :- function greet() { // code; };

★ Operators :-

Arithmetic Operators:-

+	=	Addition
-	=	Subtraction
*	=	Multiply.
/	=	Division
%	=	Modulus/remainder
**	=	Exponent

Assigment Operators:-

=	:-	x = 5
+=	:-	x += 5 i.e. x = x+5;
-=	:-	x -= 5; i.e x = x-5;
*=	:-	x *= 5 x = x * 5;
/=	:-	x /= 5 x = x/5;

Logical Operators :-

① && :- AND

② || :- OR.

③ ! - NOT

Page No.
Date

★ Comparison Operators:-

1. == :- Value checks :- 5==5 // True.
2. === :- Value & type :- 5==='5' // True.
3. != :- Value check :- 5!=3 // True.
4. !== :- Value & type :- 5!=='3' // false
5. > :- Greater :- 5>3 // True.
6. < :- Smaller :- 5<10 // True.
7. >= :- Greater / Equal :- 5>=5 // true
8. <= :- Smaller / Equal :- 5<=5 // true.

Comparison operator always returns value in true or false.

★ Conditional Statement:-

1. if

```
if ( condition) {
    code;
}
```

2. if else

```
if ( condition){
    code ;
} else {
    code ;
}
```

3. if else if

```
if (condition) {
    code;
} else if (condition) {
    code;
} else {
    statement;
}
```

```
switch ( ){
  case 1;
    code
    break;
  case 2;
    code
    break
  default;
    code;
}
```

Javascript

Page No.
Date

★ What is javascript ?
- It is programing language.
- It is used make web pages interactive and dynamic.
- It is used to add logic and behaviour.
- It is dynamic because : it can change automatically without reload or restarting.
- User interaction allowed.
- Dynamic [change content based on action and data]

★ What is let and const :- [ES6 2015].

let :- Block-scoped { }
- It is used declare variables.
- Values can be changed
- Redeclaration not possible / allowed in the same scope.

ex :-
```
{ let a = 10;
  let a = 20; // Error.
}
```
Not allowed :

```
if (true) {
  let x = 20;
}
console.log(x); // Error not valid out of the scope.
```
Accessible only inside the scopes.

Var :-
- It is also used for declare variable.
- It is old way of declaration
- It is a function-scoped variable declaration.
- It can redeclare or reassign.

ex :-
```
var a = 20;
a = 21;
console.log(a);
```
- Reassign or redeclaration is allowed.

```
var name = "Atharva";
var name = "Aniket";
console.log(name);
```
↳ Aniket;

↳ It is accessible throughout the function

Features	let	Var
Scope	block	function
redeclare	Not allowed	allowed.
Modern use	Recommended.	Avoid

★ Const:-

↳ const keyword used to declare variables whose value cannot be reassign.

↳ It is used for fixed values.

↳ It is block-scope variable declaration.

↳ Initialization is must after declaring const.

↳

★ function:-

↳ function keyword is used for function definition.

↳ function can be stored in variable

↳ It cannot be called before definition.

Example:-
```
function greet(){
    console.log("Hello");
}
```

```
function add(a,b){
    console.log(a+b);
}
add(5,3);
```

→ It can be called before definition.

Page No.
Date

This func stored in add → anonymous function / function without name.

```
const add = function (a,b) {
    return a+b;
};
```
} function expression.

This pass as argument/parameter to the function. then the function is execute and then it returns the value

```
console.log (add(5,3));
```

★ Function Expression:- / Anonymous function.

↳ It is function stored inside the variable.

↳ This function is anonymous / function without name.

↳ This function is executed using variable. or passed as argument.

↳ result of anonymous function is stored in variable

example:- → Anonymous function stored in variable 'add'.

```
const add = function(a,b) {
    return a+b;
};
let result = add(5,3);
console.log(result);
```

This will store the value which return

→ Calls the function with parameters. a & b.

↳ This function has its own this.

↳ This function is called using variable name.

★ Arrow function:-

↳ This function is a short and modern way to write function.

↳ This function is defined using (=>) symbol.

↳ It is introduced in ES6 (2015)

Syntax:-

```
const function name = () => {
    // code
};
```

Page No.
Date

Example:-

① const add = (a, b) => {
    return a+b;
};

② const add = (a, b) => a+b;
if there is only one line in code then there is no need of { } and return

★ Default parameter.
↳ Default parameters allow function to use the predefine value for parameters. when no argument is pass.
↳ default parameter values assigned to function is called without arguments.

Examples:-

① function greet(name = "Everyone") {
    console.log("Hello" + name);
};
greet(); // Hello Everyone
greet("Avi"); // Hello Avi

② function add(a=0, b=0){
    return a+b;
}
add(); // 0 a=0, b=0
add(3); // 3 a=3, b=0
add(5,3); // 8. a=5, b=3.

Page No.
Date

★ return values:-

↳ return value is a value that function sends back to the place where it was called.

↳ return keyword is used to return

↳ return stop function execution.

Examples:-

```
function add(a,b){          ⟵ It returns the value where it is called.
    return a+b;
}
let result = add(5,3);      ⟵ This calls function add(5,3).
console.log(result);        ⟵ This stores the value in result.
// 8.                       ⟵ The output is
```

★

```
function show(){
    console.log("Hello");
}
let x = show();             ⟵ "does is not returned any value
console.log(x);
                            ⟵ // undefined.
```

★

```
function age(){
    return 10;
}
let age = age();            ⟵ When it calls the return value is stored in variable age.
console.log(age);
                            ⟵ o/p :- 10.
```

Page No.
Date

★ Objects:-

- An object is a collection of data stored in key-value pairs.
- Object creation means creating a variable that can store multiple related data using properties and methods.
- Syntax:-

(1) Using key-value pair.

```
const obj_name = {
    key1 = value1;
    key2 = value2;
}
```

(2) Using new Object():

```
const obj = new Object();
obj.key1 = value1;
obj.key2 = value2;
```

(3) Using constructor:-

```
function class_name/constructor_name(param1, param2){
    this.property1 = param1;
    this.property2 = param2;
}

const p1 = new Person("abc", 21);
const p2 = new Person("xyz", 23);

console.log(p1.name); // abc
console.log(p2.age); // 21
```

Page No.
Date

↳ We can create copies of objects: or
↳ we can create separate objects.

Example:-

①
```
const Student{
    name: "Atharva";
    age : 24;
};
const S1 = new Student();
const S2 = new Student("Rahul", 23);
// Accessing each property of object
console.log (S1.name);  // Atharva
console.log (S2.age);   // 23
```

② Using Constructor function.
```
function Student (name, age){
    this.name = name;
    this.age = age;
}
const S1 = new Student("Atharva", 24);
const S2 = new Student("Rahul", 23);
    console.log(S1.name); // Atharva
    console.log(S1.age);  // 24.
```

Page No.
Date

★ Array & Array Methods.

↳ Special variable used to multiple values

```
const nums = [10,20,30,40];
```

① Create new Array :-

```
const arr_name = [ Value1, Value2, Value3];
```

② Using new Array ();

```
const arr_name = new Array( Value1, Value2);
```

Array methods.

① map();-

↳ returns new array.

↳ original remains unchanged.

Example;-

```
const nums = [ 10,20,30];
const new = nums.map( n => n*2);
```

② filter();-

↳ used to select elements in multiple elements.

↳ It returns new Arrays.

↳ It returns element who satisfy the condition.

```
const nums = [10,20, 30];
const new = nums.filter( n => n>10);
```

Page No.
Date

③ find ();-

↳ It returns single value.
↳ stops when match is found.
↳ It stops when it found its first match

```
const nums = [1, 2, 3, 4];
const found = nums.find(n => n === 2); // 2.
```

④ reduce ();-

↳ It return number, string, object.
↳ It convert or reduce in single value.

```
nums = [1, 2, 3, 4];
const total = nums.reduce((total, n) => total + n, 0);
```

⑤ some ();-

↳ It checks at least one element match condition.
↳ It returns true / false.

```
const nums = [1, 2, 3, 4];
const hasEven = nums.some(n => n % 2 === 0);
```

⑥ every ();-

↳ It checks if all elements are satisfy the condition.
↳ It return true / false.

```
const nums = [2, 4, 6];
const allEven = nums.every(n => n % 2 === 0);
```